

Air University



Operating Systems (Lab)

Year 2026

Ms. Qurrat Ul Ain

Academic Year: 2024 – 2028
Department of Cyber Security

USAMA ARSHAD | 247141
BS – Cyber Security – Fall – 24 (A)
Date of Submission: May 08, 2026

Lab # 08

Threads with Joins

1. File Creation

```
usama@ubuntu:~/Desktop$ nano threads_join.c
```

2. Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>      /* sleep() */

/* Thread function: prints 0 to 4 */
void *thread_func(void *arg)
{
    for (int i = 0; i <= 4; i++) {
        printf("[Thread]  %d\n", i);
        fflush(stdout);
        sleep(1);
    }
    return NULL;
}

int main(void)
{
    pthread_t tid;

    printf("Main thread started.\n\n");

    /* Create the thread */
    if (pthread_create(&tid, NULL, thread_func, NULL) != 0) {
        perror("pthread_create failed");
        exit(EXIT_FAILURE);
    }

    /* Main prints 20 to 24 */
    for (int i = 20; i <= 24; i++) {
        printf("[Main]    %d\n", i);
        fflush(stdout);
        sleep(1);
    }

    /* Wait for the thread to finish before exiting */
    pthread_join(tid, NULL);

    printf("\nMain thread finished. Thread has completed.\n");
    return 0;
}
```

3. Permission & Compilation

```
usama@ubuntu:~/Desktop$ chmod +x threads_join.c
usama@ubuntu:~/Desktop$ gcc thread_join.c -o thread_join -lpthread

cc1: fatal error: thread_join.c: No such file or directory
compilation terminated.
usama@ubuntu:~/Desktop$ gcc threads_join.c -o threads_join -lpthread
```

4. Output

```
usama@ubuntu:~/Desktop$ ./threads_join
Main thread started.

[Thread] 0
[Main] 20
[Main] 21
[Thread] 1
[Thread] 2
[Main] 22
[Main] 23
[Thread] 3
[Thread] 4
[Main] 24

Main thread finished. Thread has completed.
```

Threads without Joins

1. File Creation

```
usama@ubuntu:~/Desktop$ nano threads.c
```

2. Code

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>      /* sleep() */

/* Thread function: prints 0 to 4 */
void *thread_func(void *arg)
{
    for (int i = 0; i <= 4; i++) {
        printf("[Thread] %d\n", i);
        fflush(stdout);
        sleep(1);
    }
    return NULL;
}

int main(void)
{
    pthread_t tid;

    printf("Main thread started.\n\n");

    /* Create the thread */
    if (pthread_create(&tid, NULL, thread_func, NULL) != 0) {
        perror("pthread_create failed");
        exit(EXIT_FAILURE);
    }

    /* Main prints 20 to 24 */
    for (int i = 20; i <= 24; i++) {
        printf("[Main] %d\n", i);
        fflush(stdout);
        sleep(1);
    }

    /* NO pthread_join() here - main returns immediately after its loop.
     * The process exits and the OS kills the still-running thread.
     * The thread will likely NOT finish printing all its numbers.      */

    printf("\nMain finished - thread may still be running but process exits NOW.\n");
    return 0;
}
```

3. Permission & Compilation

```
usama@ubuntu:~/Desktop$ chmod +x threads.c  
usama@ubuntu:~/Desktop$ gcc threads.c -o threads -lpthread
```

4. Output

```
usama@ubuntu:~/Desktop$ ./threads  
Main thread started.  
  
[Thread] 0  
[Main] 20  
[Thread] 1  
[Main] 21  
[Main] 22  
[Thread] 2  
[Main] 23  
[Thread] 3  
[Thread] 4  
[Main] 24  
  
Main finished – thread may still be running but process exits NOW.
```

Observation:

#	With pthread_join() (./threads_join)	Without pthread_join() (./threads)
1	All main numbers (20,21,22,23,24) are printed.	Main number 22 is missing (only 20,21,23,24 appear).
2	The final message says: “Main thread finished. Thread has completed.” → indicates the child thread finished before the program exited.	Final message: “Main finished - thread may still be running but process exits NOW.” → child thread was cut off because the process terminated.
3	The order of interleaving is: Thread 0, Main 20, Main 21, Thread 1, Thread 2, Main 22, Main 23, Thread 3, Thread 4, Main 24.	Order is: Thread 0, Main 20, Thread 1, Main 21, Thread 2, Main 23, Thread 3, Thread 4, Main 24 – different pattern and missing Main 22.
4	The main thread waits for the child thread to finish, so all 5 thread numbers (0–4) appear.	The child thread may be terminated prematurely – here all 5 thread numbers still appeared (by chance), but the main loop was interrupted before printing 22.
5	The program’s exit is clean and predictable – all output is flushed.	The output is incomplete (missing 22) because the process terminated while the printf buffers or loop iterations were not yet completed.